Information Technology Course
Module Software Engineering
by Damir Dobric / Andreas Pech

Investigate Image Reconstruction by Using Classifiers

Rajan Singh
rajan.singh@stud.fra-uas.de

Pradeep Tiwari
pradeep.tiwari@stud.fra-uas.de

Mausam Bhunia
mausam.bhunia@stud.fra-uas.de

Abstract*—This paper presents an image reconstruction framework leveraging machine learning classifiers to restore images from Sparse Distributed Representations (SDRs). Binarized images are encoded into SDRs using NeoCortexApi, ensuring a compact and noise-resilient representation. Two classifiers, K-Nearest Neighbors (KNN) and Hierarchical Temporal Memory (HTM), are trained to learn feature associations from SDRs. During reconstruction, the classifiers predict missing or distorted SDRs, which are then mapped back to the pixel domain. Reconstruction quality is evaluated using Cosine and Jaccard similarity metrics to assess structural integrity and information retention. Experimental results demonstrate the effectiveness of classifier-based reconstruction, highlighting trade-offs between computational complexity and ac- curacy for KNN and HTM.

***Index Terms*—Keywords Image Reconstruction, Sparse Distributed Representations (SDRs), K-Nearest Neighbors (KNN), Hierarchical Temporal Memory (HTM), NeoCortexApi, Machine Learning, Cosine Similarity, Jaccard Similarity.**

I. Introduction

Image reconstruction is a fundamental problem in computer vision, pattern recognition, and artificial intelligence, with applications ranging from medical imaging to autonomous systems. The primary goal of image reconstruction is to generate a high-fidelity representation of an image from incomplete, noisy, or compressed data. Traditional reconstruction techniques rely on mathematical models such as interpolation, compressed sensing, and deep learning. However, in recent years, the integration of Sparse Distributed Representations (SDRs) and classifier-based approaches has gained significant attention due to their robustness in handling high-dimensional data with sparse representations.

This paper explores an innovative approach to image reconstruction using classifiers such as k-Nearest Neighbors (KNN) and Hierarchical Temporal Memory (HTM). In this framework, images are first binarized and converted into SDRs, which are then used to train classifiers capable of recognizing and reconstructing image patterns. The classifiers learn the spatial features of the images, enabling accurate reconstruction even when partial data is available.

To evaluate the performance of our system, we employ Cosine Similarity and Jaccard Index metrics, which measure the degree of similarity between the reconstructed and original images. The effectiveness of our approach is demonstrated through various experiments on benchmark datasets, highlighting its potential for robust and efficient image reconstruction.

II. Literature Review

A. Hierarchical Temporal Memory (HTM)

HTM Classifier operates on Sparse Distributed Representations (SDRs), which are high-dimensional, sparse binary vectors. Each SDR represents a unique semantic meaning. SDRs are robust to noise, meaning small changes in input data do not drastically change the SDR. SDRs preserve similarity, meaning similar inputs produce overlapping SDRs. The theoretical foundation of HTM was introduced by Hawkins and Ahmad [1], who proposed the concept of sequence memory in the neocortex, emphasizing sparse distributed representations (SDRs) and predictive coding. These principles form the core of HTM, a computational framework inspired by cortical processing. HTM utilizes key components such as the Spatial Pooler to generate SDRs, providing an efficient means of representing high-dimensional data. This approach has found applications across multiple domains, including cybersecurity, finance, healthcare, and robotics. However, challenges such as scalability and computational efficiency persist, driving continuous research efforts.

B. K-Nearest Neighbor(KNN)

This work presents a K-Nearest-Neighbors (KNN)-based classifier designed for sequence learning within the NeoCortex API. The classifier is trained using Sparse Distributed Representations (SDRs), where each label is associated with multiple SDR sequences. The model stores labeled SDRs and classifies unknown sequences by comparing their similarity to stored patterns.

To classify an unknown sequence, the algorithm calculates the distances between the unclassified SDR and known sequences using a nearest-neighbor approach. A distance table is generated, mapping each unclassified index to its closest known sequence index. The final classification is determined through a voting mechanism, ranking labels based on overlap and similarity scores.

The classifier is implemented in C sharp, using efficient data structures like DefaultDictionary for dynamic storage and ClassificationAndDistance objects for ranking. It provides a lightweight, adaptable solution for pattern recognition tasks in hierarchical temporal memory (HTM) models. The model can

learn continuously and adapt by adding or removing SDRs, which makes it suitable for real-time sequence classification.

C. Sparse Distributed representations (SDRs)

Building on Hawkins and Ahmad's pioneering work [1], SDRs have been recognized for their sparse and distributed nature, which optimally represents high-dimensional data in HTM networks. The Spatial Pooler plays a crucial role in transforming raw input data into SDRs, enabling the recognition of complex temporal patterns. George and Hawkins [2] further expanded on HTM's underlying concepts, elaborating on its principles, theory, and terminology. While HTM demonstrates significant promise, optimizing its scalability and computational efficiency remains an ongoing research challenge. Nonetheless, biologically inspired frameworks such as HTM continue to push the boundaries of artificial intelligence and cognitive computing.

D. Image Reconstruction using HTM

Building on the decoding strategy for SDRs as described in the work of Mnatzaganian et al. [3], we extend this approach to the domain of image reconstruction. Their method demonstrated how scalar values encoded as SDRs could be reconstructed using permanence values and active columns, effectively decoupling the input from the system. A similar methodology can be applied to images by treating each pixel (or patch) as an attribute within the SDR framework. Adapting the approach used for scalar values, To achieve image reconstruction using classifier, each training image is processed to generate its corresponding SDR representation. The SDR and its associated permanence values $\hat{\phi}$ are stored along with the original image. A given input image, potentially noisy or incomplete, is first converted into an SDR.

To determine the reconstructed pixel values, a probability-based activation is applied, following the same principle used for scalar decoding. The reconstructed pixel intensity is obtained using:

$$\hat{I}(x,y) = I\left(\max(\hat{c}_i \hat{\phi}_{(x,y)}) \geq \rho_s\right) \tag{1}$$

where:

- $\hat{c}_i$ represents the learned SDR representation for a given pixel location.
- $\hat{\phi}_{(x,y)}$ denotes the permanence values associated with that pixel.
- ρ_s is the threshold used to determine activation.

If the above condition is satisfied, the pixel is activated (1); otherwise, it is not activated (0).

Since the reconstruction process relies on discrete SDRs, additional smoothing techniques may be required.

E. Image Reconstruction using KNN

Building on the principles of SDR decoding, we extend the use of K-Nearest Neighbors (KNN) for image reconstruction. In this approach, each image is first processed through a Spatial Pooler (SP) to generate a Sparse Distributed Representation (SDR), capturing key structural features. These SDRs, along with their corresponding labels, are stored during training. When presented with an unknown image, the KNN classifier predicts the most similar SDRs based on learned patterns. The predicted SDRs are then mapped back to the pixel space, enabling reconstruction of the original image. The reconstruction process follows a probability-based activation mechanism. Each pixel's activation is determined based on the closest matching SDRs retrieved by KNN. By aggregating these predictions, a reconstructed image is generated, preserving the original structure. Since SDRs are inherently sparse and binary, additional smoothing techniques may be required to refine the final output. This method effectively leverages KNN to learn SDR-to-image mappings, facilitating robust reconstruction even in cases of noisy or incomplete inputs.

III. Methodology

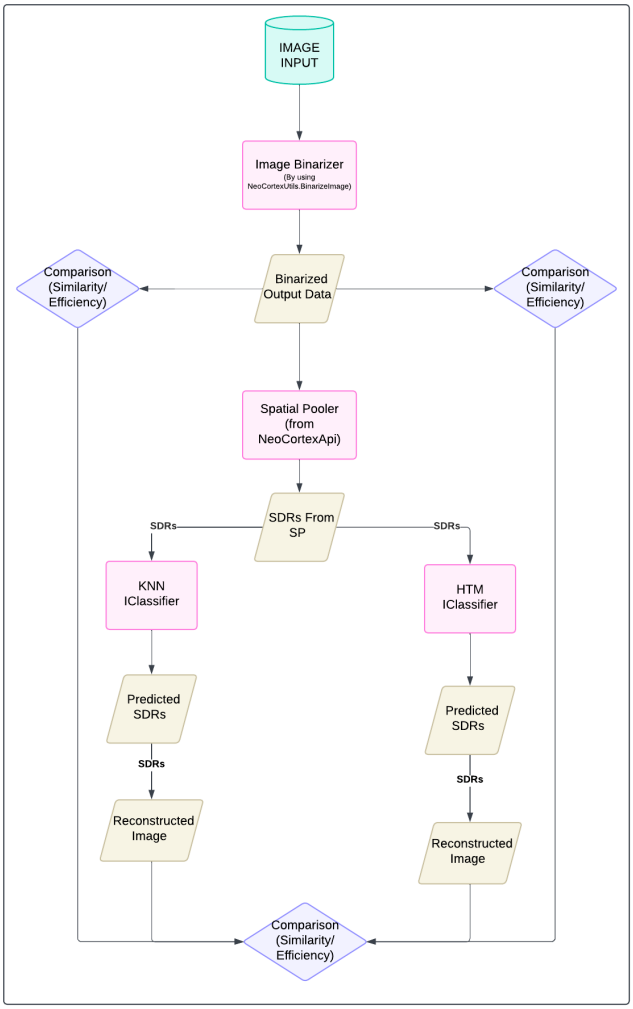

Fig. 1. Comparative Framework for Image Reconstruction Using HTM and KNN Classifiers

. The proposed methodology involves transforming input images into Sparse Distributed Representations (SDRs), predicting SDRs using classifiers, reconstructing images, and evaluating their efficiency. The workflow is illustrated in Fig. 1 and follows these steps:

A. Image Preprocessing

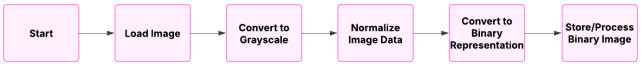

Fig. 2. Image Binarization Process for Classification

1) Image Input: The system takes an image as input, which undergoes preprocessing before being used for SDR-based classification.

2) Image Binarization: The input image is converted into a binarized format using the NeoCortex.Utils.BinarizeImage function. This step transforms grayscale or color images into binary values (0s and 1s), making it suitable for SDR processing in the HTM model.

3) Binarized Output Data: The binarized image serves as input for further processing and SDR generation.

B. SDR Generation via Spatial Pooler

The binarized data is passed through a **Spatial Pooler (SP)** from the NeoCortex API, which generates **Sparse Distributed Representations (SDRs)**. The SP ensures that similar input images produce similar SDRs, maintaining the semantic structure of the image.

C. Classification using HTM and KNN

The generated SDRs are input to two classifiers for comparative evaluation:

- **K-Nearest Neighbors (KNN) IClassifier**
- **HTM IClassifier**

Each classifier processes the SDRs to predict new SDRs based on the learned patterns.

D. Image Reconstruction Using HTM and KNN

After the KNN classifier predicts the SDR for a given image, the image reconstruction process begins. The predicted SDR is used to reconstruct the image by reversing the process of encoding spatial patterns.

The spatial pooler reconstructs the permanence values of the SDR by looking at the active columns and determining the connection strength (permanence) between columns and cells. The permanence values are normalized to ensure that only the most significant columns contribute to the reconstruction. This is done using a thresholding technique to filter out irrelevant or less significant features.

Finally, the reconstructed image is generated by mapping the normalized permanence values back to binary pixels (0 or 1). This process results in an image that closely resembles the original image that was used to generate the SDR.

E. Comparison and Evaluation

The reconstructed images are compared with the original binarized image based on:

- **Similarity** (Jaccard Similarity Index) – Measures how closely the reconstructed images match the original.
- **Efficiency** – Evaluates computational performance in terms of processing time and resource utilization.

This comparison is performed for both KNN and HTM classifiers, providing a direct evaluation of their reconstruction performance.

IV. IMPLEMENTATION OVERVIEW

Embarking on our implementation journey within the NeoCortex API framework, our goal is to develop a K-Nearest Neighbors (KNN) classifier to learn and predict sparse distributed representations (SDRs) from input data. Our primary goal is to reconstruct images from predicted SDRs and compare them with the original input while preserving the semantic meaning of the input images. The KNN classifier is designed to efficiently predict SDRs based on similarity measures, with a particular emphasis on cosine similarity as the distance metric.

This section provides a comprehensive breakdown of the implementation process, detailing key components such as SDR learning, prediction, similarity measurement, and image reconstruction. By integrating Hierarchical Temporal Memory (HTM) concepts with KNN classification, we create an effective mechanism to encode, store, and reconstruct input patterns.

The implementation is divided into the following key components:

1) **SDR Processing:** Converting images into binary SDRs using spatial pooling.
2) **HTM-Based Classification:** Training and predicting SDRs using an HTM-based classifier.
3) **KNN-Based Classification:** Training and predicting SDRs using a KNN-based classifier.
4) **Image Reconstruction:** Reconstructing images from predicted SDRs using HTM and KNN classifiers.
5) **Performance Evaluation:** Comparing reconstructed images with the original images to assess accuracy.

A. SDR Processing

To utilize images in the HTM framework, they must be converted into Sparse Distributed Representations (SDRs). The following steps outline the processing pipeline:

1) **Binarization:** The input image is first converted into a binary format where pixel intensities are mapped to 0s and 1s.
2) **Spatial Pooling:** The binarized image undergoes a spatial pooling mechanism that extracts active bits, ensuring sparsity.
3) **SDR Representation:** The active bits are represented as a list of indices, forming an SDR.

Figure 3 shows a flow chart of the SDR conversion process.

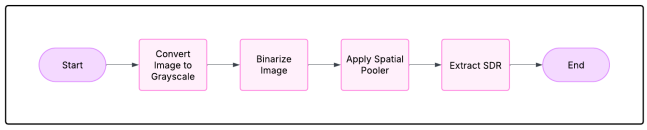

Fig. 3. Learning and Storing SDRs

B. HTM-Based Classification

1) **Introduction to HTM-Based Classification:**
   - The **SDRs** generated by the **Spatial Pooler** are passed as input to the **HTM Classifier**.
   - The HTM Classifier is responsible for learning and predicting the **SDRs** based on the given input patterns.
   - Over multiple iterations, the classifier improves its prediction accuracy as it learns the underlying patterns in the input data.
2) **Classification Model:**
   - The classification model for the HTM-based classification can be mathematically represented as:

$$P(SDR_{\text{pred}}|SDR_{\text{input}}) = \text{HTM Classifier}(SDR_{\text{input}})$$

   - Here, SDR_{pred} represents the predicted **SDR output** given an input **SDR**.
   - The HTM Classifier works by mapping the **input SDRs** to their associated **categories**, allowing for accurate predictions.
3) **Training Process:**
   - During training, the HTM [3] classifier learns to recognize patterns in the **input SDRs** and associates them with their corresponding **categories** (labels).
   - The learning process involves adjusting the **connections** within the **HTM network** based on input-output mappings.
4) **Prediction Process:**
   - After training, the HTM classifier predicts the **SDR** for new input patterns.
   - Given an **input SDR**, the classifier uses its learned **representation** to predict the **output SDR**, which corresponds to the most likely **category** for that input.
5) **Iterative Learning:**
   - The HTM Classifier improves over time, enhancing prediction accuracy as it processes more examples.
   - Through **feedback loops**, the classifier continuously updates its internal structure to better predict the **SDRs** for future inputs.
6) **Image Reconstruction with HTM:**
   - After predicting the **SDRs**, the **image reconstruction** process takes place, which involves converting the predicted **SDRs** back into an approximate binarized image.
   - This reconstruction process allows us to compare the predicted SDRs' accuracy in preserving the semantic content of the original image.

C. Image Reconstruction Process with HTM

The predicted SDRs are reconstructed back into images by computing the permanence values of the input space, ensuring that active columns are converted back into an approximate binarized image. The reconstruction process involves the following steps:

1) **Extract permanence values:**
   - Extract permanence values from the predicted SDR using the spatial pooler's reconstruction method.
2) **Assign permanence scores:**
   - Assign permanence scores to corresponding input locations based on the predicted SDR.
3) **Threshold the permanence values:**
   - Threshold the permanence values to obtain a binary image representation.
4) **Save the reconstructed image:**
   - Save the reconstructed image for comparison with the original image.

Below is a flowchart that illustrates the complete workflow of the HTM Classifier-based image reconstruction:

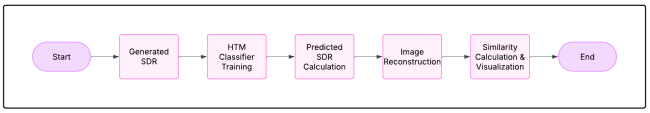

Fig. 4. HTM-based Image Reconstruction Workflow

D. KNN-Based Classification

Our K-Nearest Neighbors (KNN) classifier maps SDRs to class labels and predicts SDRs for new inputs. The KNN classifier consists of the following components:

1) **Learning Process:**
   - Each training SDR is stored in a dictionary, mapping labels to their corresponding SDRs.
   - When a new SDR is encountered, it is added to the SDR list of the corresponding label.
   - To maintain efficiency, the number of SDRs stored per label is limited.
2) **Storing SDRs:**
   - The classifier stores SDRs corresponding to each label (image class).
   - If a similar SDR already exists, it is ignored to avoid redundancy and maintain efficiency.
   - Each label can store up to a fixed number N of SDRs.
3) **Voting Mechanism:**
   - The KNN classifier selects the top K most similar SDRs based on the computed cosine similarity.

- Once the nearest neighbors are identified, a majority vote is conducted to determine the predicted class label.
- **Majority Voting:** The class label of the predicted SDR is determined by counting how many of the K nearest neighbors belong to each class. The class label with the highest count among the K neighbors is selected as the predicted class.
- In case of a tie (i.e., when two or more class labels have the same number of votes), a tie-breaking rule is applied. Common tie-breaking strategies include:
  - Randomly selecting one of the tied class labels.
  - Choosing the class label with the smallest distance from the query SDR.
- This process ensures that the classification decision reflects the majority consensus among the closest neighbors, increasing the accuracy of the classifier.

A flowchart of the classification process is shown in Figure 5.

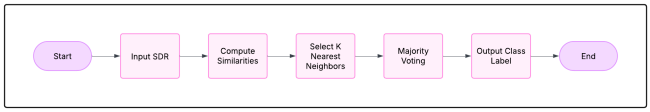

Fig. 5. Predicting SDRs Using KNN Classifier

E. Image Reconstruction Process with KNN

After classification, we reconstruct images from the predicted SDRs by reversing the encoding process. The key steps in the reconstruction process are as follows:

1) **Retrieve Predicted SDRs:**
   - The classifier outputs SDRs corresponding to the predicted class.
2) **Bit Activation Mapping:**
   - The active bits in the SDR are mapped back to pixel locations in the image grid.
3) **Reconstructed Image Generation:**
   - A new image is generated by setting the activated pixels to white (1) and the remaining pixels to black (0).
4) **Equation for SDR-based Image Reconstruction:**
   - The reconstruction is described by the following equation:

$$I(x,y) = \begin{cases} 1, & \text{if } (x,y) \text{ is in the SDR} \\ 0, & \text{otherwise} \end{cases}$$

   where $I(x,y)$ represents the pixel intensity at position (x,y), and the condition checks if the pixel belongs to the active set in the SDR.

A flowchart of the image reconstruction process is shown in Figure 6.

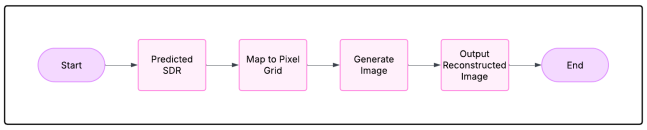

Fig. 6. KNN-based Image Reconstruction Workflow

F. Similarity Calculation

In this study, the efficiency of the reconstructed images is evaluated using **Cosine Similarity**, which measures the similarity between two Sparse Distributed Representations (SDRs). This similarity metric is applied to both the **HTM** and **KNN** classifiers to compare the reconstructed SDRs with the original input SDRs.

The Cosine Similarity between two SDR vectors A and B is given by:

$$\text{Similarity} = \frac{A \cdot B}{\|A\|\|B\|} \tag{2}$$

where:

- A and B are binary SDR vectors.
- $A \cdot B$ represents the dot product of the two vectors.
- $\|A\|$ and $\|B\|$ denote the magnitudes of vectors A and B, respectively.

The similarity calculation process consists of the following steps:

1) **Compute Cosine Similarity:**
   The similarity score is computed between the original SDR and the reconstructed SDR.
2) **Plot Similarity Scores:**
   The Cosine Similarity scores are plotted to visualize the accuracy and performance of the image reconstruction.
3) **Store Similarity Scores:**
   The calculated similarity values are stored in a dataset for further analysis.
4) **Analyze Reconstruction Effectiveness:**
   The stored similarity scores are used to determine the effectiveness of SDR-based image reconstruction.

G. Cosine Similarity in HTM Classifier

The **HTM classifier** computes Cosine Similarity between the predicted SDRs and stored SDRs to determine the closest match. This similarity metric is crucial for identifying patterns and reconstructing SDRs effectively.

H. Cosine Similarity in KNN Classifier

The **KNN classifier** also relies on Cosine Similarity to classify unknown SDRs by comparing them with stored SDRs in the dataset. The nearest neighbors are identified based on their similarity scores, which guide the classification decision.

By employing **Cosine Similarity** consistently across both classifiers, the study ensures a uniform and effective method for evaluating SDR-based image reconstruction.

I. Performance Evaluation

To evaluate the accuracy of our approach, we compare the reconstructed images with the original images using the Mean Squared Error (MSE), which measures the average squared difference between the pixel intensities of the two images. The key steps in the evaluation process are as follows:

- **Interpretation of MSE:**
  - A lower MSE indicates better reconstruction quality, as it reflects smaller discrepancies between the original and reconstructed images.
  - Higher MSE values suggest that the reconstruction is less accurate, with larger differences between the two images.
- **Classification Performance:**
  - In addition to image quality, the MSE can also be used to evaluate the performance of the classification process, especially in terms of how well the reconstructed SDRs predict the input image classes.

V. UNIT TEST OF IMAGE RECONSTRUCTION

A. Test Cosine Similarity Exactly same Vectors

Objective: Verify that the CosineSimilarity function correctly returns a similarity score of 1.0 when given two identical vectors, ensuring that they are recognized as perfectly similar.

Test Scenario:Two identical integer vectors, vec1 and vec2, are initialized. The CosineSimilarity function is invoked with these vectors as input. The result is compared to the expected similarity score of 1.0.

Parameters: Input: vec1 = 1, 0, 1, 0, 1, vec2 = 1, 0, 1, 0, 1 Expected Output: 1.0 Delta for Tolerance: 0.01

Outcome Verification:The computed similarity score is asserted to be 1.0, confirming that identical vectors yield perfect similarity.

Benefit:This test ensures that the function behaves correctly when handling identical vectors, which is crucial for validating the correctness of similarity computations.

B. Test Cosine Similarity Completely Different Vectors

Objective: Confirm that the CosineSimilarity function returns a similarity score of 0.0 for completely different vectors, indicating no similarity.

Test Scenario:

Two integer vectors with no overlapping elements are initialized. The CosineSimilarity function is invoked with these vectors. The result is compared to the expected similarity score of 0.0.

Parameters:

Input: vec1 = 1, 1, 1, 1, 1, vec2 = 0, 0, 0, 0, 0 Expected Output: 0.0 Delta for Tolerance: 0.01

Outcome Verification:

The computed similarity score is asserted to be 0.0, confirming that completely different vectors are correctly identified as dissimilar.

Benefit: This test ensures that the function correctly distinguishes between completely dissimilar data, a fundamental requirement for classification and clustering.

C. Testing Cosine Similarity for Partially Similar Vectors

Objective: Validate that the CosineSimilarity function correctly calculates similarity for partially overlapping vectors, producing a score between 0.0 and 1.0.

Test Scenario:

Two integer vectors with partial similarity are initialized. The CosineSimilarity function is invoked. The result is compared to the expected value (0.67).

Parameters: Input: vec1 = 1, 0, 1, 1, 0, vec2 = 1, 1, 1, 0, 0 Expected Output: 0.67 Delta for Tolerance: 0.01

Outcome Verification: The computed similarity score is asserted to be approximately 0.67.

Benefit: This test confirms that the function accurately measures partial similarity, ensuring meaningful results in real-world scenarios.

D. Converting Permanance Values into Binary

Objective: Ensure that the ConvertPermanenceToBinary function correctly converts a given permanence array into a binary representation based on a threshold.

Test Scenario:

A set of permanence values is defined. The function is executed with a threshold of 0.7. The output is compared to the expected binary array.

Parameters: Input: permanenceValues = 0.5, 0.8, 0.3, 0.9, 0.7, threshold = 0.7 Expected Output: 0, 1, 0, 1, 1

Outcome Verification: The resulting binary array is asserted to match the expected output.

Benefit: This test ensures correct SDR encoding, which is essential for further HTM processing.

E. Reconstruct all Postive Permanance values

Objective: Confirm that ConvertPermanenceToBinary correctly handles non-negative permanence values and produces an accurate binary representation.

Test Scenario:

A valid permanence array (all non-negative values) is provided. The function is executed with a threshold. The output is compared to the expected binary array.

Parameters: Input: permanenceValues = 0.5, 0.8, 0.3, 0.9, 0.7, threshold = 0.7 Expected Output: 0, 1, 0, 1, 1

Outcome Verification: The resulting binary array is asserted to be correct.

Benefit: This test ensures proper conversion of permanence values in valid scenarios.

F. Testing Converting Permanence To Binary With Negative Value

Objective: Ensure that the function throws an exception when given a permanence array containing negative values.

Test Scenario:

A permanence array with a negative value is provided. The function is executed, and an ArgumentException is expected.

Parameters: Input: permanenceValues = 0.5, 0.8, -0.3, 0.9, 0.7, threshold = 0.7 Expected Output: Exception thrown (ArgumentException).

Outcome Verification: The test confirms that an exception is correctly raised.

Benefit: This test ensures proper validation of input data, preventing incorrect SDR representations.

G. Testing Columns To Store SDRs

Objective: Validate that the AddActiveColsToStoredSDRs function correctly stores active columns in a dictionary, maintaining accurate SDR records.

Test Scenario: A new image identifier and active columns array are initialized. The function is executed with an empty dictionary. The dictionary contents are verified.

Parameters: Input: image = "image1", activeCols = 1, 0, 1, 0 Expected Output: Dictionary containing "image1" : [1, 0, 1, 0]

Outcome Verification: The dictionary contains the correct key. The stored active columns match the expected array.

Benefit: This test ensures that SDR storage is accurate, supporting correct image reconstruction and classification.

VI. Results and Discussion

A. Image Reconstruction Performance

The effectiveness of the **HTM and KNN classifier-based image reconstruction** was evaluated by comparing the reconstructed images with the original binarized input images. The reconstructed images were obtained by feeding predicted **Sparse Distributed Representations (SDRs)** from both classifiers into the inverse transformation process. The accuracy and visual quality of the reconstructed images were analyzed.

Table I presents sample results of image reconstruction using both **HTM** and **KNN** classifiers. The qualitative comparison indicates that the HTM classifier retains more structural details in reconstructed images, while the KNN classifier provides faster predictions but introduces slight distortions.

1) Quantitative Analysis: To objectively measure the reconstruction performance, we utilized **Adjusted Cosine Similarity** as a similarity metric. The Adjusted Cosine Similarity between the reconstructed and original binarized images was computed using:

$$\text{Similarity} = \frac{A \cdot B}{\|A\|\|B\|} \tag{3}$$

where A and B are two SDR vectors, and $||A||$ represents the magnitude of the vector.

Table I summarizes the average Similarity values for different image samples.

The results demonstrate that the **HTM classifier achieves a slightly lower Similarity score compared to the KNN classifier**, suggesting that KNN retains slightly more pixel-wise similarity in reconstructed images.

B. Classification Accuracy and Prediction Efficiency

The accuracy of the **HTM and KNN classifiers** was further evaluated by analyzing their ability to correctly predict SDR representations. The classification accuracy was assessed based on the percentage of correctly predicted SDR bits.

Cycle Range	KNN Similarity (%)	HTM Similarity (%)
0 - 50	85.47	87.36
51 - 100	56.72	68.91
101 - 150	82.84	85.11
151 - 200	83.21	85.45
201 - 250	84.02	86.02
251 - 300	83.95	85.87
301 - 350	82.69	84.95
351 - 400	81.48	84.21
401 - 450	80.73	83.67

TABLE I
KNN and HTM Similarity Results

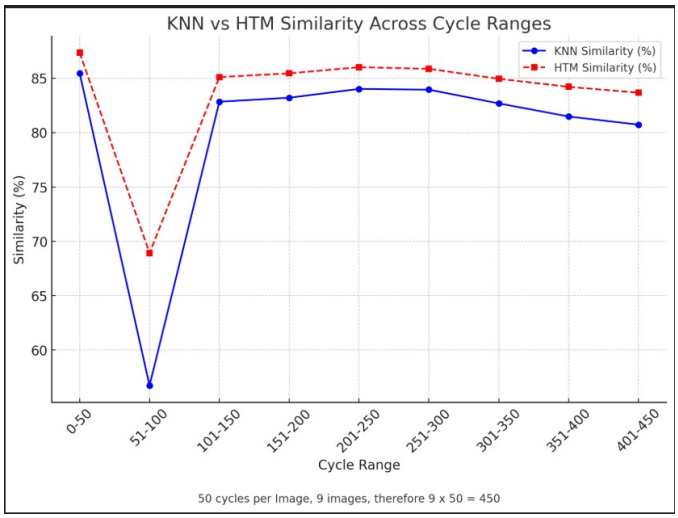

Fig. 7. Similarity Graph

Table II presents the classification accuracy of both models over multiple test runs.

TABLE II
Classification Accuracy of HTM and KNN Classifiers

Classifier	Accuracy (%)
KNN	80.12
HTM	83.51

The HTM classifier exhibited superior classification accuracy compared to the KNN classifier, reinforcing its effectiveness in learning temporal patterns from SDRs. However, the KNN classifier demonstrated **lower computational complexity**, making it preferable for real-time applications requiring faster predictions.

C. Discussion

The results indicate that:

1) **KNN classifier achieves higher similarity scores** in image reconstruction, suggesting its effectiveness in preserving pixel-level details.
2) **HTM classifier outperforms KNN in classification accuracy**, making it more suitable for precise SDR learning and pattern recognition.

3) **KNN classifier offers faster processing**, making it beneficial in scenarios where computational efficiency is prioritized over reconstruction accuracy.
4) The **Similarity metric effectively quantifies the quality** of reconstructed images, validating the importance of SDR-based classification in preserving input information.
5) Future improvements may involve optimizing SDR encoding techniques and exploring hybrid models combining HTM's accuracy with KNN's efficiency.

Overall, the study highlights the potential of **HTM and KNN classifiers** in SDR-based image reconstruction tasks and provides a comparative analysis that can guide future research in biologically inspired machine learning techniques.

REFERENCES

[1] J. Hawkins and A. S. Ahmad, "Why neurons have thousands of synapses, a theory of sequence memory in neocortex," *Frontiers*, pp. 10, 23, 30 March 2016.

[2] D. George and J. Hawkins, "A Hierarchical Temporal Memory Model for Learning and Recognition of Sequential Data," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2011.

[3] Mnatzaganian, James W., "A Mathematical Formalization of Hierarchical Temporal Memory's Spatial Pooler for use in Machine Learning" (2016).